Manual de Apoio — UFCD 5425

Projeto de Tecnologias e Programação de Sistemas de Informação

UFCD 5425 — 50 horas

Manual de apoio, com anexos de exemplos de projeto, fases e ferramentas de organização

Formador: Eduardo Fernandes

Índice

(em Word: clique com o botão direito sobre o índice e escolha "Atualizar campo" para preencher os números de página)

Introdução 6

Para quem é este manual 6

Porque é que isto continua a ser difícil 6

Como usar os quatro exemplos deste manual 6

Estrutura da UFCD e deste manual 7

Parte I — Conceitos fundamentais 7

1. O que é um projeto? 7

1.1 Uma definição que vale a pena conhecer 7

1.2 Projeto, tarefa do dia a dia, e processo contínuo 8

1.3 A restrição tripla: âmbito, tempo e custo 8

1.4 Um perigo com nome próprio: o "scope creep" 9

2. O que é um Sistema de Informação? 9

2.1 Mais do que um programa 9

2.2 Porque é que isto importa para o projeto 10

2.3 Um exemplo do dia a dia 10

3. Visão geral das fases de um projeto de TPSI 10

3.1 Porque faz sentido dividir um projeto em fases 10

3.2 As fases não são estanques 11

Parte II — As seis fases, em detalhe 11

4. Fase 1 — Definição 11

4.1 O que esta fase resolve 11

4.2 Identificar o problema ou a necessidade 11

4.3 Escrever objetivos SMART 12

4.4 Definir o público-alvo 12

4.5 Levantamento de requisitos: o que o sistema deve fazer 12

4.6 Erros comuns nesta fase 13

5. Fase 2 — Planeamento 13

5.1 O que esta fase resolve 13

5.2 Decompor o trabalho em partes mais pequenas 13

5.3 Calendarizar o trabalho 14

5.4 Identificar recursos e riscos 14

5.5 Critérios de sucesso 14

6. Fase 3 — Elaboração / Desenvolvimento 15

6.1 O que esta fase resolve 15

6.2 Desenhar antes de construir 15

6.3 Construir de forma incremental 15
6.4 Organizar o código à medida que cresce 15
7. Fase 4 — Testes 16
7.1 Porque testar 16
7.2 Tipos de teste 16
7.3 Testar também os casos estranhos 17
7.4 Registar o que foi testado 17
8. Fase 5 — Apresentação 17
8.1 O que esta fase resolve 17
8.2 Preparar a demonstração 17
8.3 Adaptar a comunicação ao público 17
8.4 Lidar com perguntas e imprevistos 18
9. Fase 6 — Elaboração do relatório 18
9.1 O que esta fase resolve 18
9.2 Estrutura sugerida 18
9.3 Documentar ao longo do caminho, não no final 18
9.4 Autoavaliação crítica 18
Conclusão 19
Referências 19
Anexo A — Os quatro exemplos de projeto 20
A.1 — ListaDeTarefas (Scratch) — nível inicial 20
Objetivo pedagógico 20
O que o programa faz 20
Estrutura do programa (resumo) 20
Conceitos de programação demonstrados 21
Ficheiro entregue 21
A.2 — ContactosFácil (HTML, CSS e JavaScript) — nível inicial/intermédio 21
Objetivo pedagógico 21
O que o programa faz 22
Trecho 1 — guardar e carregar os dados 22
Trecho 2 — adicionar um contacto (validação simples) 22
Trecho 3 — proteção contra código indesejado (escaparTexto) 22
Ficheiros entregues 22
A.3 — TarefaFácil (Python, Flask e SQLite) — nível intermédio 23
Objetivo pedagógico 23
O que o programa faz 23
Trecho 1 — o modelo de dados (3 tabelas relacionadas) 23

Trecho 2 — proteger páginas que exigem sessão iniciada 23
Trecho 3 — deteção automática de tarefas atrasadas 24
Como executar 24
Ficheiros entregues 24
A.4 — AgendaC (C) — nível avançado 24
Objetivo pedagógico 25
O que o programa faz 25
Trecho 1 — a estrutura de dados (struct) de um contacto 25
Trecho 2 — copiar texto em segurança 25
Trecho 3 — gravar os dados em ficheiro 26
Como compilar e executar 26
Ficheiros entregues 26
A.5 — Comparação final 26
Anexo B — Fases do projeto: tabelas e documentos para preencher 28
B.1 — Ficha de Definição do Projeto (Fase 1) 29
1. Qual é o problema ou a necessidade que este projeto resolve? 29
2. Objetivos do projeto (critério SMART — ver secção 4.3) 29
3. Público-alvo 29
4. Lista inicial de requisitos (secção 4.5) 29
5. O que este projeto NÃO vai incluir 30
B.2 — Ficha de Planeamento (Fase 2) 31
1. Estrutura de tarefas (WBS simplificada — secção 5.2) 31
2. Calendário (modelo de Gantt simplificado para as 22 sessões) 31
3. Recursos necessários 31
4. Riscos identificados 32
5. Critérios de sucesso 32
B.3 — Diário de Bordo do Desenvolvimento (Fase 3) 33
B.4 — Ficha de Testes (Fase 4) 33
1. Testes funcionais (secção 7.2) 33
2. Testes de casos-limite e de erro (secção 7.3) 33
3. Teste de usabilidade 34
B.5 — Guião de Apresentação (Fase 5) 35
1. Passos da demonstração (secção 8.2) 35
2. Problema que o projeto resolve (frase de abertura) 35
3. Plano B (o que fazer se algo correr mal ao vivo) 35
4. Perguntas previsíveis e respostas preparadas 35
B.6 — Modelo de Relatório Final (Fase 6) 36

Anexo C — Sugestões de ferramentas de organização 37
C.1 — Para a fase de Definição e Planeamento 38
C.2 — Para a fase de Elaboração / Desenvolvimento 38
C.3 — Para a fase de Testes 39
C.4 — Para a fase de Apresentação 39
C.5 — Para a fase de Relatório 39
C.6 — Uma nota final sobre ferramentas 40

Introdução

Para quem é este manual

Este manual destina-se a alunos que estão a desenvolver um projeto no âmbito da UFCD 5425. Foi escrito para servir dois públicos ao mesmo tempo, porque é isso que normalmente encontramos numa turma real:

- quem está a dar os primeiros passos em programação e precisa de perceber, de forma concreta e sem pressa, o que é um projeto e como se organiza um trabalho deste tipo do início ao fim;
- quem já tem mais experiência e vai usar este mesmo guião para planear e documentar um projeto final mais exigente.

Por isso, cada conceito é explicado a partir da base — nenhuma ideia é apresentada como "óbvia" — mas é sempre acompanhado de uma aplicação mais exigente, para que o manual continue a ser útil mesmo quando o projeto cresce em complexidade.

Porque é que isto continua a ser difícil

É muito comum, ao longo desta UFCD, surgir a mesma dificuldade: perceber o que é, de facto, "um projeto" — e, mais concretamente, saber organizar cada uma das suas fases. Isso não acontece porque o tema seja pouco claro; acontece porque, antes deste momento, a maior parte dos trabalhos escolares consistiu em tarefas fechadas, com um enunciado e uma entrega — e um projeto real pede outra coisa: que o próprio aluno decida o que vai construir, para quem, com que recursos, e que organize o seu próprio percurso até lá chegar. Este manual trata exatamente disso — não apenas do "como programar", mas do "como organizar o trabalho de programar".

Como usar os quatro exemplos deste manual

No **Anexo A** encontram-se quatro versões de um exercício semelhante — uma pequena aplicação de gestão de informação (contactos ou tarefas) — construídas em quatro tecnologias diferentes, por ordem crescente de complexidade:

1. **Scratch** (programação em blocos) — o ponto de partida ideal para quem está a dar os primeiros passos, ou para introduzir rapidamente os conceitos de variável, lista, ciclo e decisão sem a barreira da sintaxe de uma linguagem de texto.
2. **HTML, CSS e JavaScript** ("ContactosFácil") — uma aplicação que corre inteiramente num browser, adequada a quem já está confortável com os conceitos básicos e quer começar a programar "a sério", numa linguagem de texto, com uma interface gráfica real.
3. **Python com Flask e base de dados** ("TarefaFácil") — um nível intermédio, que introduz um servidor, contas de utilizador, e dados guardados de forma permanente numa base de dados.

4. **C** ("AgendaC") — o exemplo mais exigente, indicado para projetos finais ou para alunos que já têm confiança com a lógica de programação e querem perceber o que acontece "por baixo" de conceitos que, nas outras linguagens, são automáticos (gestão de memória, ficheiros binários, estruturas de dados).

Não é obrigatório usar todos os quatro. O mais importante é escolher o nível certo para o ponto em que cada aluno está — e, se for caso disso, avançar de um nível para o seguinte ao longo do curso, usando sempre a mesma lógica de "gerir uma lista de coisas" como fio condutor.

Estrutura da UFCD e deste manual

A UFCD 5425 tem a duração de 45 horas e organiza-se em seis fases, que correspondem também à estrutura das seis secções principais deste manual:

Fase	Objetivo principal
1. Definição	Decidir, com clareza, o que vai ser construído e para quem
2. Planeamento	Organizar o trabalho no tempo e nos recursos disponíveis
3. Elaboração / Desenvolvimento	Construir a solução, passo a passo
4. Testes	Verificar que a solução funciona como previsto
5. Apresentação	Comunicar o trabalho realizado
6. Elaboração do relatório	Documentar e refletir sobre todo o processo

No final deste manual, o **Anexo B** disponibiliza tabelas e documentos para preencher em cada uma destas fases, e o **Anexo C** sugere ferramentas (gratuitas e simples) que ajudam a organizar o trabalho.

Parte I — Conceitos fundamentais

1. O que é um projeto?

1.1 Uma definição que vale a pena conhecer

A palavra "projeto" usa-se para quase tudo no dia a dia — "o meu projeto de férias", "um projeto de lei" — mas, em gestão e em engenharia, tem um significado preciso. O *Project Management Body of Knowledge* (PMBOK), publicado pelo Project Management Institute (PMI, 2021), define projeto como:

> *"um esforço temporário, empreendido para criar um produto, serviço ou resultado único."*

A norma internacional ISO 21500 (2012), sobre orientações para a gestão de projetos, apresenta uma definição muito próxima: um conjunto único de processos, com datas de início e de fim definidas, realizado para atingir um objetivo.

Destas duas definições, que são hoje as referências mais usadas em todo o mundo, retiram-se três características que **todo** o projeto tem — e que servem de teste simples para saber se algo é, de facto, um projeto:

1. **É temporário.** Tem um início e, mais importante ainda, tem um fim previsto. Quando o objetivo é alcançado (ou se deixa de fazer sentido continuar), o projeto termina.
2. **Cria algo único.** O resultado — um produto, um serviço, ou até um relatório — não existia exatamente assim antes. Pode usar peças já conhecidas (uma linguagem de programação, uma biblioteca), mas o resultado final é novo.
3. **Tem um objetivo definido.** Existe uma razão clara para o projeto existir, e será essa razão que vai permitir, no final, dizer se o projeto foi bem-sucedido ou não.

1.2 Projeto, tarefa do dia a dia, e processo contínuo

Uma forma simples de fixar esta ideia é compará-la com duas outras que lhe são próximas, mas diferentes:

- **Uma tarefa do dia a dia** — como lavar a loiça ou enviar um email — tem um início e um fim, mas não cria nada de único: repete-se, de forma muito parecida, vezes sem conta.
- **Um processo contínuo** — como o funcionamento de uma linha de produção numa fábrica, ou a gestão diária de um restaurante — não tem um fim planeado: a ideia é que continue a funcionar indefinidamente.
- **Um projeto** — como organizar a festa de aniversário de um amigo, construir uma casa, ou desenvolver a aplicação que vai ser entregue nesta UFCD — tem um fim à vista desde o primeiro dia, e o resultado final (a festa, a casa, a aplicação) é, de alguma forma, único.

Esta distinção interessa particularmente aqui porque um erro comum, no início de um projeto de programação, é tratá-lo como se fosse apenas "mais uma tarefa" — sentar-se a escrever código sem primeiro decidir o que se vai construir nem até onde se vai chegar. É precisamente para evitar isso que existem as fases de Definição e Planeamento, abordadas nas secções 4 e 5.

1.3 A restrição tripla: âmbito, tempo e custo

Todo o projeto é limitado por três fatores que estão ligados entre si — o chamado **triângulo de ferro** ou **restrição tripla** da gestão de projetos:

- **Âmbito** (ou *scope*): tudo o que o projeto vai efetivamente fazer — e, por omissão, tudo o que vai deixar de fazer.
- **Tempo**: quanto tempo há disponível até à data de entrega.
- **Custo**: que recursos existem — no caso de um projeto escolar, isto traduz-se sobretudo em tempo de trabalho disponível e nas ferramentas (muitas vezes gratuitas) a que se tem acesso.

A ideia central deste triângulo é que os três lados estão relacionados: se se decidir aumentar o âmbito (fazer mais funcionalidades) sem aumentar o tempo disponível, alguma coisa tem de ceder — normalmente, a qualidade do que é entregue. Um projeto bem gerido é aquele em que estas três variáveis foram equilibradas de forma consciente, e não por acidente, na véspera da entrega.

Muitas referências mais recentes (incluindo a 7.ª edição do PMBOK, PMI, 2021) acrescentam uma quarta dimensão a este triângulo: a **qualidade** — o grau em que o resultado final cumpre efetivamente o que foi pedido. Para o efeito deste manual, basta reter a ideia principal: âmbito, tempo e recursos empurram-se mutuamente, e a qualidade é o que normalmente paga a fatura quando este equilíbrio não é bem gerido.

1.4 Um perigo com nome próprio: o "scope creep"

Em gestão de projetos, chama-se *scope creep* (literalmente, "fuga do âmbito") ao fenómeno de o âmbito de um projeto ir crescendo pouco a pouco, sem uma decisão explícita — "já agora, também podia ter...", "e se acrescentássemos...". Cada pequeno acrescento, isolado, parece inofensivo; o problema é a soma de todos eles ao longo do tempo, que acaba por pôr em risco o prazo de entrega de tudo o resto.

A melhor defesa contra o *scope creep* é exatamente o que este manual defende desde o início: escrever, na fase de Definição, o que o projeto vai e não vai incluir — e voltar a esse documento sempre que surgir a tentação de "acrescentar só mais uma coisinha".

---

2. O que é um Sistema de Informação?

2.1 Mais do que um programa

É fácil confundir "sistema de informação" com "programa de computador" ou "aplicação" — mas o conceito é mais amplo do que isso. Segundo Laudon e Laudon (2020), autores de referência na área dos sistemas de informação de gestão, um **sistema de informação** é:

> *um conjunto de componentes interrelacionados que recolhem, processam, armazenam e distribuem informação, com o objetivo de apoiar a tomada de decisão, a coordenação, o controlo, a análise e a visualização numa organização.*

O ponto fundamental desta definição é que um sistema de informação não é apenas o código ou o programa: é a combinação de **quatro componentes**, que têm de funcionar em conjunto:

1. **Pessoas** — quem usa o sistema e quem o mantém;
2. **Processos** — as regras e os procedimentos que organizam como a informação é usada (por exemplo, "só o professor pode editar as notas");
3. **Dados** — a informação propriamente dita (os contactos, as tarefas, os registos);
4. **Tecnologia** — o software e o hardware que tornam tudo isto possível.

2.2 Porque é que isto importa para o projeto

Esta distinção tem uma consequência prática muito concreta para qualquer projeto desta UFCD: **o código é apenas uma parte do trabalho.** Um projeto bem definido também pensa em quem vai usar a aplicação (pessoas), em que regras devem existir (processos — por exemplo, "cada utilizador só vê as suas próprias tarefas", como acontece no exemplo TarefaFácil, no Anexo A) e em como os dados devem estar organizados (por exemplo, decidir que cada contacto tem nome, telefone e email, e nada mais).

Os quatro exemplos do Anexo A são, todos eles, pequenos sistemas de informação completos — ainda que simples — precisamente porque, em cada um, existe uma decisão sobre dados (o que é guardado), sobre processos (o que se pode fazer com esses dados) e sobre pessoas (quem os vai usar), além do código que os torna possíveis.

2.3 Um exemplo do dia a dia

Pense-se numa agenda de papel, usada por uma secretária de uma clínica para marcar consultas. Mesmo sem nenhum computador envolvido, já existe ali um sistema de informação: há dados (os nomes, as horas), há um processo (a forma como as consultas são marcadas e confirmadas), há pessoas (a secretária, os médicos, os pacientes) e há uma "tecnologia" — neste caso, o caderno e a caneta. Substituir esse caderno por uma aplicação (como o TarefaFácil, no Anexo A) não muda a natureza do sistema — apenas a tecnologia que o suporta. É um exemplo útil para perceber que **programar não é criar algo do nada: é, quase sempre, informatizar algo que já existia de outra forma.**

---

3. Visão geral das fases de um projeto de TPSI

3.1 Porque faz sentido dividir um projeto em fases

Tanto o PMBOK (PMI, 2021) como a norma ISO 21500 (2012) organizam a gestão de um projeto em **grupos de processos** que se repetem, com nomes diferentes mas uma lógica semelhante em praticamente qualquer metodologia de gestão de projetos: começar por definir o que se vai fazer, planear como o fazer, executá-lo, controlá-lo ao longo do caminho, e encerrá-lo formalmente no final.

As seis fases desta UFCD seguem exatamente esta lógica, adaptada à escala de um projeto de programação realizado por um aluno (ou por um pequeno grupo), ao longo de 45 horas:

Fase da UFCD 5425	Grupo de processos equivalente (PMBOK / ISO 21500)
1. Definição	Iniciação
2. Planeamento	Planeamento
3. Elaboração / Desenvolvimento	Execução

Fase da UFCD 5425	Grupo de processos equivalente (PMBOK / ISO 21500)
4. Testes	Monitorização e controlo
5. Apresentação	Encerramento (parte 1)
6. Elaboração do relatório	Encerramento (parte 2)

3.2 As fases não são estanques

Um mal-entendido comum é pensar que estas fases têm de ser rigorosamente sequenciais — terminar completamente a Definição antes de tocar no Planeamento, terminar completamente o Planeamento antes de escrever a primeira linha de código, e assim sucessivamente. Na prática, mesmo em metodologias mais tradicionais, há sempre alguma sobreposição: é normal voltar atrás e ajustar o planeamento depois de, já na fase de Desenvolvimento, se perceber que algo era mais difícil (ou mais fácil) do que se pensava.

O que **não** deve acontecer é avançar para o Desenvolvimento sem nunca ter passado, ainda que rapidamente, pela Definição e pelo Planeamento — porque é precisamente isso que leva à dificuldade identificada no início deste manual: alunos que começam a programar sem saber bem o que estão a construir, nem até onde têm de chegar.

As secções seguintes detalham cada uma destas seis fases.

Parte II — As seis fases, em detalhe

4. Fase 1 — Definição

4.1 O que esta fase resolve

A fase de Definição responde a uma pergunta que parece simples mas que raramente tem uma resposta óbvia à primeira tentativa: **o que é, exatamente, que vamos construir?** Esta fase termina quando essa pergunta tem uma resposta escrita, clara e partilhada — não apenas "uma ideia na cabeça".

4.2 Identificar o problema ou a necessidade

Todo o projeto parte de um problema real (ou, pelo menos, de uma necessidade real) que o resultado final vai resolver. Antes de pensar em tecnologia, vale a pena responder, em poucas frases, a estas perguntas:

- Que dificuldade concreta é que este projeto vai resolver?
- Quem sente essa dificuldade hoje, e como a resolve atualmente (se é que a resolve)?

- O que vai mudar, na prática, quando o projeto estiver concluído?

Por exemplo: o problema que está na origem do exemplo TarefaFácil (Anexo A) não é "fazer um programa em Python" — é "uma pessoa com várias tarefas e compromissos perde-se facilmente se não tiver um sítio único onde os organizar". A tecnologia (Python, Flask, uma base de dados) é apenas o meio escolhido para resolver esse problema.

4.3 Escrever objetivos SMART

Depois de identificado o problema, o passo seguinte é transformá-lo em objetivos concretos. Uma das ferramentas mais usadas para isto, desde que foi proposta por Doran (1981), é o critério **SMART** — um objetivo bem escrito deve ser:

- **Specific / Específico** — diz exatamente o que vai ser feito, sem ambiguidade;
- **Measurable / Mensurável** — é possível verificar, de forma objetiva, se foi ou não alcançado;
- **Achievable / Atingível** — é realista, dado o tempo e os recursos disponíveis;
- **Relevant / Relevante** — está mesmo ligado ao problema identificado;
- **Time-bound / Temporal** — tem um prazo definido.

Compare-se um objetivo mal escrito com a sua versão SMART:

> ✗ *"Fazer uma aplicação de tarefas boa e completa."*
>
> ☑ *"Até ao final da UFCD, construir uma aplicação web onde um utilizador registado pode criar, editar, concluir e eliminar tarefas com data-limite, e ver um calendário semanal com os seus eventos."*

A segunda versão permite, no final do projeto, verificar objetivamente se foi cumprida ou não — a primeira não.

4.4 Definir o público-alvo

Um sistema de informação (secção 2) existe para ser usado por pessoas concretas. Definir quem são essas pessoas — ainda que de forma simples — ajuda a tomar decisões ao longo de todo o projeto: a linguagem usada na interface, o grau de simplicidade necessário, e até que funcionalidades são realmente prioritárias.

Perguntas úteis nesta fase:

- Quem vai usar esta aplicação? (por exemplo: "estudantes", "a equipa de um pequeno café", "eu próprio, para uso pessoal")
- Que conhecimentos técnicos é razoável assumir que essas pessoas têm?
- Vão usar a aplicação num computador, num telemóvel, ou em ambos?

4.5 Levantamento de requisitos: o que o sistema deve fazer

Com o problema, os objetivos e o público definidos, falta ainda decidir, com algum detalhe, o que o sistema deve e não deve fazer. A esta lista chama-se, em engenharia de software, **levantamento de requisitos**, e costuma dividir-se em dois tipos:

- **Requisitos funcionais** — o que o sistema faz. Exemplo: "o utilizador pode adicionar um contacto com nome, telefone e email."
- **Requisitos não funcionais** — como o sistema o faz, ou que qualidades deve ter. Exemplo: "a lista de contactos deve continuar visível mesmo depois de fechar e voltar a abrir o browser" (como acontece no ContactosFácil, graças ao armazenamento local, no Anexo A).

Não é necessário, numa UFCD deste tipo, produzir um documento de requisitos tão extenso como o de um projeto profissional — mas listar, mesmo que em poucas linhas, "o que a aplicação vai fazer" é o que permite, mais tarde, saber se o projeto está, de facto, concluído.

4.6 Erros comuns nesta fase

- **Começar a programar antes de terminar esta fase.** É talvez o erro mais frequente — e o que mais tempo acaba por custar, porque leva a reescrever trabalho já feito quando, mais tarde, se percebe que a ideia inicial não estava bem definida.
- **Definir um âmbito demasiado ambicioso** para o tempo disponível (ver secção 1.3 sobre a restrição tripla).
- **Confundir a solução com o problema** — por exemplo, decidir "vou usar inteligência artificial" antes de saber qual é, realmente, o problema a resolver.

O Anexo B inclui um documento para preencher nesta fase (a "Ficha de Definição do Projeto"), com espaço para o problema, os objetivos SMART, o público-alvo e a lista inicial de requisitos.

---

5. Fase 2 — Planeamento

5.1 O que esta fase resolve

Se a Definição respondeu a "o quê", o Planeamento responde a "como" e "quando": como vai o trabalho ser dividido, em que ordem, e quanto tempo vai cada parte ocupar.

5.2 Decompor o trabalho em partes mais pequenas

A ferramenta clássica para isto, em gestão de projetos, chama-se **Estrutura Analítica do Projeto** (do inglês *Work Breakdown Structure*, ou WBS) — uma forma de dividir um projeto grande em partes cada vez mais pequenas, até chegar a tarefas que são fáceis de estimar e de executar.

Na prática, para um projeto desta UFCD, isto significa transformar um objetivo como "construir a aplicação de tarefas" numa lista de tarefas concretas, por exemplo:

1. Desenhar a base de dados (que tabelas, que colunas)
2. Construir o registo e o login de utilizadores

3. Construir a criação e listagem de tarefas
4. Construir a edição e eliminação de tarefas
5. Construir o calendário semanal
6. Testar tudo
7. Preparar a apresentação

Cada uma destas tarefas é já pequena e concreta o suficiente para ser planeada individualmente — e é exatamente esta lista, com estimativas de tempo associadas, que constitui o plano do projeto.

5.3 Calendarizar o trabalho

Depois de identificadas as tarefas, o passo seguinte é distribuí-las no tempo disponível. Não é necessário usar uma ferramenta complexa: um simples **diagrama de Gantt** — um quadro em que cada linha é uma tarefa e cada coluna é uma semana (ou uma sessão de aulas), com uma barra a indicar quando essa tarefa vai ser feita — é suficiente, e é uma das formas de calendarização mais usadas em projetos de qualquer dimensão desde que foi popularizada por Henry Gantt, no início do século XX. O Anexo B inclui um modelo deste tipo de calendário, já adaptado às 22 sessões desta UFCD.

Uma recomendação importante ao calendarizar: **reservar sempre algum tempo extra**, sobretudo perto do fim, para imprevistos e para a fase de Testes — é extremamente comum subestimar o tempo que esta fase realmente exige.

5.4 Identificar recursos e riscos

Além do tempo, vale a pena listar, ainda nesta fase:

- **Recursos necessários** — que ferramentas, equipamento ou conhecimentos o projeto vai exigir, e se já estão disponíveis;
- **Riscos previsíveis** — o que pode correr mal, e o que fazer se isso acontecer. Por exemplo: "se não conseguir terminar o calendário semanal a tempo, a aplicação continua a funcionar sem essa funcionalidade" — ter sempre, sempre que possível, um "plano B" simples para as partes mais arriscadas do projeto.

5.5 Critérios de sucesso

Por fim, esta fase deve deixar claro **como se vai saber que o projeto foi bem-sucedido** — idealmente, revisitando os objetivos SMART definidos na fase anterior e transformando-os numa pequena lista de verificação. Esta lista será mais tarde muito útil na fase de Testes (secção 7) e na fase de Relatório (secção 9).

6. Fase 3 — Elaboração / Desenvolvimento

6.1 O que esta fase resolve

É nesta fase que a solução é efetivamente construída — mas construir bem não significa simplesmente "começar a escrever código". Esta fase inclui também o desenho da solução, antes de qualquer linha de código, e uma forma organizada de a ir construindo.

6.2 Desenhar antes de construir

Antes de programar, é muito útil desenhar, ainda que de forma simples, como a solução vai funcionar. Duas ferramentas particularmente úteis, mesmo em esboço, num projeto deste nível:

- **Esboços de interface** (*wireframes*) — desenhos simples, à mão ou num programa de desenho, de como cada ecrã ou página vai estar organizado. Ajudam a decidir, antes de programar, onde vai ficar cada botão, cada campo, cada lista.
- **Modelo de dados** — uma lista simples das "coisas" que o sistema vai guardar e dos seus atributos. Por exemplo, no TarefaFácil (Anexo A), o modelo de dados inclui um "Utilizador" (com nome, email e palavra-passe), uma "Tarefa" (com título, prazo e estado) e um "Evento" (com título, data e hora) — e a relação entre eles (cada tarefa pertence a um utilizador). Este tipo de representação é uma versão simplificada dos diagramas de classes e de entidade-relacionamento usados na modelação UML (*Unified Modeling Language*, mantida pela Object Management Group), uma notação comum em engenharia de software para descrever a estrutura de um sistema antes de o construir.

6.3 Construir de forma incremental

Em vez de tentar construir toda a aplicação de uma só vez, a abordagem mais segura — e a mais usada profissionalmente, sob diferentes nomes (desenvolvimento iterativo, incremental, ágil) — é dividir a construção em pequenos incrementos que já funcionam, testando cada um antes de avançar para o seguinte. Por exemplo, na construção do TarefaFácil (Anexo A), a ordem seguida foi:

1. Primeiro, a base de dados e o registo/login — sem esta base, nada do resto faz sentido;
2. Depois, a criação e listagem de tarefas — a funcionalidade central;
3. Só depois, a edição, a conclusão e a eliminação de tarefas;
4. Por fim, o calendário semanal — uma funcionalidade adicional, que podia ter sido deixada de lado se o tempo se esgotasse, sem pôr em risco o resto da aplicação.

Esta ordem não é arbitrária: começa sempre pelo que é indispensável, e só depois acrescenta o que é importante mas não crítico — a mesma lógica que, na fase de Planeamento, levou a reservar tempo extra para imprevistos.

6.4 Organizar o código à medida que cresce

Os quatro exemplos do Anexo A partilham uma preocupação comum, visível na forma como os ficheiros estão organizados: **separar responsabilidades diferentes em sítios diferentes.**

- No ContactosFácil, a estrutura (HTML), a aparência (CSS) e o comportamento (JavaScript) estão em três ficheiros distintos.
- No TarefaFácil, os dados (`modelos.py`), as rotas (`rotas.py`) e as páginas (`templates/`) estão também separados.
- No AgendaC, cada tipo de informação (contactos, tarefas) tem o seu próprio par de ficheiros `.c`/`.h`, independente da gestão de ficheiros em disco.

Esta separação — por vezes chamada *separação de responsabilidades* — não é uma formalidade: é o que torna possível continuar a trabalhar num projeto que cresce, sem que cada pequena alteração obrigue a perceber (ou a arriscar estragar) todo o resto do código.

---

7. Fase 4 — Testes

7.1 Porque testar

Testar um sistema de informação é verificar, de forma deliberada e organizada, que ele faz o que devia fazer — e que não faz o que não devia. Não é um passo opcional nem uma formalidade de fim de projeto: é, muitas vezes, a fase que revela os problemas que, se não forem corrigidos a tempo, acabam por aparecer precisamente durante a apresentação final.

7.2 Tipos de teste

A literatura de engenharia de software (ver, por exemplo, Sommerville, 2015) distingue vários tipos de teste, que se podem aplicar, em maior ou menor grau, a qualquer um dos quatro exemplos do Anexo A:

- **Teste unitário** — verifica uma única "peça" do sistema de forma isolada. Por exemplo, confirmar que a função `tarefa_esta_atrasada()` do TarefaFácil devolve `True` apenas quando o prazo já passou e a tarefa não está concluída.
- **Teste de integração** — verifica que várias peças funcionam bem em conjunto. Por exemplo, confirmar que, depois de criar uma tarefa através do formulário, ela aparece corretamente na lista e fica guardada na base de dados.
- **Teste funcional** (ou de sistema) — verifica que uma funcionalidade completa, do ponto de vista de quem usa a aplicação, funciona do início ao fim. Por exemplo: registar uma conta, iniciar sessão, criar uma tarefa, marcá-la como concluída.
- **Teste de usabilidade** — verifica não só se a aplicação funciona, mas se é fácil e agradável de usar. Por exemplo, pedir a alguém que nunca viu a aplicação que tente usá-la sem ajuda, e observar onde hesita ou se engana.

Não é necessário, num projeto desta UFCD, escrever testes automatizados complexos (ainda que, para quem quiser ir mais além, seja uma excelente forma de aprofundar o exemplo em Python). O essencial é **testar de forma deliberada e registada** — e não apenas "experimentar uma vez e assumir que está tudo bem".

7.3 Testar também os casos estranhos

Uma armadilha comum é testar apenas o "caminho feliz" — isto é, testar só aquilo que acontece quando tudo corre como esperado. Um projeto bem testado verifica também o que acontece nos casos-limite e nos erros previsíveis: o que acontece se o formulário for submetido sem preencher um campo obrigatório? O que acontece se um utilizador tentar aceder a uma página sem ter sessão iniciada (como é explicitamente verificado no TarefaFácil, através do decorador `login_necessario`)? O que acontece se a lista estiver vazia?

7.4 Registar o que foi testado

É boa prática manter um registo simples dos testes realizados — o que foi testado, o resultado, e o que foi corrigido. O Anexo B inclui um modelo de "Ficha de Testes" para este efeito. Este registo é também uma prova concreta, a apresentar no relatório final (fase 6), de que o projeto foi efetivamente verificado, e não apenas "entregue e esperado que funcione".

---

8. Fase 5 — Apresentação

8.1 O que esta fase resolve

Um projeto só está, de facto, concluído quando o trabalho realizado é comunicado a outras pessoas — e a forma como essa comunicação é feita influencia diretamente como o trabalho vai ser avaliado, independentemente da qualidade técnica por trás dele.

8.2 Preparar a demonstração

Uma apresentação eficaz mostra a aplicação a funcionar, em vez de apenas descrever o que ela faz. Algumas recomendações práticas:

- Preparar previamente um pequeno **guião** com os passos exatos que vão ser demonstrados (por exemplo: registar um utilizador, criar duas tarefas, mostrar o calendário) — e praticá-lo pelo menos uma vez antes da apresentação real.
- Ter sempre um **plano B** caso algo corra mal ao vivo: por exemplo, ter capturas de ecrã ou um pequeno vídeo de apoio, para o caso de a ligação à internet ou o equipamento falharem no momento.
- Começar pelo **problema que o projeto resolve**, antes de mostrar a solução — a audiência compreende e valoriza muito melhor uma funcionalidade quando sabe que problema real ela resolve.

8.3 Adaptar a comunicação ao público

A mesma informação deve ser comunicada de forma diferente a um público técnico (que vai querer saber como as coisas foram feitas) e a um público não técnico (que quer sobretudo entender o que a aplicação faz e porque é útil). Vale a pena preparar, mentalmente, as duas versões — e ouvir com atenção quem está a avaliar, para perceber qual delas é mais adequada no momento.

8.4 Lidar com perguntas e imprevistos

É normal surgirem perguntas para as quais não se tem uma resposta imediata. Responder com honestidade ("não pensei nesse caso, mas a forma como o código está organizado permitiria acrescentá-lo facilmente") é sempre preferível a inventar uma resposta — e mostra, muitas vezes, mais maturidade técnica do que fingir saber tudo.

---

9. Fase 6 — Elaboração do relatório

9.1 O que esta fase resolve

O relatório é o registo escrito de todo o percurso do projeto — da Definição à Apresentação — e é frequentemente a parte do trabalho que permite a quem avalia (e ao próprio aluno, mais tarde) compreender não só o que foi construído, mas também **porquê** e **como**.

9.2 Estrutura sugerida

Uma estrutura simples e eficaz para este tipo de relatório técnico inclui, tipicamente:

1. **Introdução** — o problema, os objetivos e o público-alvo (retomando a fase de Definição);
2. **Planeamento** — como o trabalho foi organizado e calendarizado;
3. **Desenvolvimento** — as principais decisões técnicas tomadas e porquê (por exemplo, "optei por guardar os dados em SQLite, em vez de num ficheiro de texto simples, porque...");
4. **Testes** — o que foi testado e com que resultado;
5. **Dificuldades encontradas e como foram resolvidas** — esta secção é, muitas vezes, a mais valorizada, porque demonstra capacidade de resolução de problemas reais;
6. **Conclusão e possíveis melhorias futuras** — o que ficou feito, e o que poderia ser a próxima versão do projeto.

O Anexo B inclui um modelo deste relatório, já estruturado, para preencher ao longo do projeto (e não apenas no último dia).

9.3 Documentar ao longo do caminho, não no final

O erro mais comum nesta fase é deixar tudo para o último momento, tentando recordar, semanas depois, decisões tomadas no início do projeto. A melhor prática — usada também em ambiente profissional — é ir registando, mesmo que em poucas linhas, as decisões importantes e as dificuldades à medida que elas acontecem, usando exatamente as tabelas do Anexo B como um "diário de bordo" do projeto.

9.4 Autoavaliação crítica

Um bom relatório não se limita a descrever o que foi feito: reflete também, com honestidade, sobre o que poderia ter corrido melhor. Esta capacidade de autoavaliação crítica é, ela própria, uma

competência central de qualquer profissional de tecnologia — ninguém entrega um projeto perfeito à primeira, e saber identificar com clareza as suas próprias limitações é o que permite melhorar no projeto seguinte.

Conclusão

Um projeto de tecnologia bem-sucedido não se mede apenas pela qualidade do código final — mede-se também pela forma como o processo, do início ao fim, foi organizado. As seis fases apresentadas neste manual (Definição, Planeamento, Elaboração/Desenvolvimento, Testes, Apresentação e Relatório) não são uma formalidade burocrática: são a tradução, para a escala de um projeto escolar, das mesmas práticas usadas em projetos profissionais de qualquer dimensão, documentadas em referências como o PMBOK (PMI, 2021) e a norma ISO 21500 (2012).

Os quatro exemplos do Anexo A mostram que esta mesma lógica — definir bem o problema, planear o trabalho, construir de forma incremental, testar, apresentar e documentar — se aplica de forma idêntica, independentemente de se estar a programar em blocos (Scratch), numa página web (HTML/JavaScript), num servidor (Python) ou mais perto do funcionamento interno de um computador (C). É esta lógica, e não uma linguagem de programação em particular, que este manual pretende, acima de tudo, ensinar.

Referências

- Doran, G. T. (1981). There's a S.M.A.R.T. way to write management's goals and objectives. *Management Review*, 70(11), 35–36.
- International Organization for Standardization. (2012). *ISO 21500:2012 — Guidance on project management*. ISO.
- Laudon, K. C., & Laudon, J. P. (2020). *Management Information Systems: Managing the Digital Firm* (16th ed.). Pearson.
- Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. OMG.
- Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (7th ed.). PMI.
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.

Anexo A — Os quatro exemplos de projeto

Este anexo apresenta os quatro exemplos referidos ao longo do manual, por ordem crescente de complexidade. Cada um resolve, com uma tecnologia diferente, um problema do mesmo género — gerir uma lista de informação (contactos ou tarefas) — o que permite comparar diretamente como os mesmos conceitos (guardar dados, perguntar, decidir, repetir) se traduzem em ferramentas diferentes.

Os quatro projetos completos (todo o código-fonte, pronto a abrir e a executar) são entregues como ficheiros separados, a acompanhar este manual. Este anexo mostra apenas os trechos mais relevantes de cada um, com comentário.

A.1 — ListaDeTarefas (Scratch) — nível inicial

Objetivo pedagógico

Introduzir, sem a barreira da sintaxe de uma linguagem de texto, os quatro conceitos que vão reaparecer em todos os outros exemplos: **variáveis**, **uma lista**, **ciclos** e **decisões**. Indicado como primeiro contacto com a programação, ou como forma rápida de discutir a lógica de um programa antes de o traduzir para uma linguagem de texto.

O que o programa faz

Ao clicar na bandeira verde, o programa pergunta repetidamente o que o utilizador quer fazer — **adicionar**, **listar**, **remover** ou **sair** — e mantém uma lista de tarefas com base nessa escolha.

Estrutura do programa (resumo)

```
quando a bandeira verde for clicada
```

Cofinanciado pela União Europeia

```
apagar tudo de [Tarefas]
definir [continuar] para (1)
repetir até <(continuar) = (0)>
  perguntar [O que quer fazer?] e esperar
  definir [escolha] para (resposta)
  se <escolha = adicionar> então
    perguntar [Qual é a tarefa?] e esperar
    acrescentar (resposta) a [Tarefas]
  se <escolha = listar> então
    (percorrer a lista e dizer cada item, numerado)
  se <escolha = remover> então
    perguntar [Qual o número?] e esperar
    apagar (resposta) de [Tarefas]
  se <escolha = sair> então
    definir [continuar] para (0)
```

A versão completa, bloco a bloco, está descrita no ficheiro `guiao_de_blocos.md`, entregue junto com o projeto `ListaDeTarefas.sb3`.

Conceitos de programação demonstrados

Conceito	Onde aparece
Variável	`escolha`, `continuar`, `i`
Lista	`Tarefas` — acrescentar, remover, obter item, comprimento
Ciclo	"repetir até", usado para o menu e para percorrer a lista
Decisão	"se ... então", um para cada opção do menu
Entrada do utilizador	"perguntar ... e esperar"

Ficheiro entregue

`ListaDeTarefas.sb3` — abrir em <https://scratch.mit.edu/create>, na aplicação Scratch Desktop, ou em <https://turbowarp.org>.

A.2 — ContactosFácil (HTML, CSS e JavaScript) — nível inicial/intermédio

Objetivo pedagógico

Primeiro contacto com uma linguagem de programação de texto, num ambiente imediatamente visível e sem necessidade de instalar nada — o próprio browser é suficiente. Introduz a manipulação da página (DOM), a reação a eventos (cliques, submissão de formulários) e a persistência de dados simples no próprio browser.

O que o programa faz

Uma página onde é possível adicionar um contacto (nome, telefone e email), ver a lista de contactos guardados, e eliminar qualquer um deles. A lista mantém-se mesmo depois de fechar e voltar a abrir o browser, graças ao `localStorage`.

Trecho 1 — guardar e carregar os dados

```
// app.js
let contactos = JSON.parse(localStorage.getItem("contactos")) || [];

function guardar() {
  localStorage.setItem("contactos", JSON.stringify(contactos));
}
```

Este é o requisito não funcional "os dados não se perdem ao fechar o browser" (ver secção 4.5 do manual) traduzido em código: o `localStorage` é uma pequena base de dados que o próprio browser disponibiliza, e que sobrevive ao fecho da página.

Trecho 2 — adicionar um contacto (validação simples)

```
function adicionarContacto(nome, telefone, email) {
  if (nome.trim() === "") {
    mostrarErro("O nome é obrigatório.");
    return;
  }
  contactos.push({ nome, telefone, email });
  guardar();
  mostrarContactos();
  limparFormulario();
}
```

Note-se a verificação do campo "nome" antes de guardar — um exemplo simples do tipo de teste de caso-limite discutido na secção 7.3 do manual: o que acontece se o utilizador tentar guardar um contacto sem nome?

Trecho 3 — proteção contra código indesejado (escaparTexto)

```
function escaparTexto(texto) {
  const div = document.createElement("div");
  div.textContent = texto;
  return div.innerHTML;
}
```

Esta função evita que um nome de contacto com símbolos especiais de HTML (por exemplo `<script>`) seja interpretado como código ao ser mostrado na página — um cuidado de segurança básico, mas importante, sempre que se mostra na página texto introduzido por um utilizador.

Ficheiros entregues

Pasta `contactosfacil/` — `index.html`, `estilo.css`, `app.js`. Abrir `index.html` diretamente num browser.

A.3 — TarefaFácil (Python, Flask e SQLite) — nível intermédio

Objetivo pedagógico

Introduzir o conceito de **servidor**: uma aplicação que corre num computador (o servidor) e é acedida através do browser, com contas de utilizador próprias e dados guardados de forma permanente numa base de dados. É o primeiro exemplo em que a palavra "sistema de informação" (secção 2 do manual) se torna visível na estrutura do próprio código: dados, processos (autenticação, regras de acesso) e pessoas (utilizadores distintos) claramente separados.

O que o programa faz

Permite registar uma conta, iniciar sessão, gerir uma lista de tarefas (com prazos e deteção automática de atraso) e um calendário semanal de eventos — cada utilizador só vê os seus próprios dados.

Trecho 1 — o modelo de dados (3 tabelas relacionadas)

```
# modelos.py
CREATE TABLE utilizadores (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
);

CREATE TABLE tarefas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT NOT NULL,
    estado TEXT NOT NULL DEFAULT 'pendente',
    prazo TEXT,
    id_utilizador INTEGER NOT NULL,
    FOREIGN KEY (id_utilizador) REFERENCES utilizadores (id) ON DELETE CASCADE
);
```

Esta é uma versão direta do "modelo de dados" discutido na secção 6.2 do manual: cada tarefa pertence a um utilizador (`id_utilizador`), e o `FOREIGN KEY ... ON DELETE CASCADE` garante que, se um utilizador for apagado, as suas tarefas são apagadas com ele — uma regra de negócio (um "processo", na definição de sistema de informação da secção 2) imposta diretamente pela base de dados.

Trecho 2 — proteger páginas que exigem sessão iniciada

```
# rotas.py
```

```
def login_necessario(funcao):
    @wraps(funcao)
    def funcao_protegida(*args, **kwargs):
        if "id_utilizador" not in session:
            flash("Inicie sessão para continuar.", "erro")
            return redirect(url_for("rotas.login"))
        return funcao(*args, **kwargs)
    return funcao_protegida

@rotas.route("/tarefas")
@login_necessario
def tarefas():
    ...
```

Este é exatamente o tipo de caso-limite discutido na secção 7.3 do manual — "o que acontece se alguém tentar aceder sem sessão iniciada?" — resolvido de forma centralizada: qualquer rota marcada com `@login_necessario` fica automaticamente protegida, em vez de repetir esta verificação em cada função.

Trecho 3 — deteção automática de tarefas atrasadas

```
def tarefa_esta_atrasada(tarefa):
    if not tarefa["prazo"]:
        return False
    if tarefa["estado"] == "concluída":
        return False
    return tarefa["prazo"] < date.today().isoformat()
```

Uma pequena função, mas um bom exemplo de requisito funcional traduzido diretamente em código (secção 4.5): uma tarefa só é "atrasada" se tiver prazo definido, esse prazo já tiver passado, e ainda não estiver concluída.

Como executar

```
cd tarefafacil
python -m venv .venv && source .venv/bin/activate
pip install -r requisitos.txt
python run.py
```

Ficheiros entregues

Pasta `tarefafacil/` — ficheiros Python (`app/`), páginas (`templates/`), estilo e comportamento (`static/`), e instruções completas em `README.md`.

A.4 — AgendaC (C) — nível avançado

Objetivo pedagógico

Mostrar o que, nas outras três linguagens, acontece de forma automática — e que, em C, tem de ser feito explicitamente: reservar espaço fixo para os dados, copiar texto com cuidado para não ultrapassar esse espaço, e gravar e ler diretamente bytes num ficheiro. Indicado para alunos que já têm confiança com a lógica de programação e querem perceber melhor o que se passa "por baixo".

O que o programa faz

Um programa de consola (sem interface gráfica) que gere contactos e tarefas, com menus de texto, e guarda os dados em ficheiros binários que são lidos automaticamente na próxima vez que o programa é executado.

Trecho 1 — a estrutura de dados (struct) de um contacto

```
// contacto.h
#define MAX_CONTACTOS 100
#define TAM_NOME 50

typedef struct {
    int id;
    char nome[TAM_NOME];
    char telefone[TAM_TELEFONE];
    char email[TAM_EMAIL];
    int ativo;
} Contacto;

typedef struct {
    Contacto itens[MAX_CONTACTOS];
    int total;
    int proximo_id;
} ListaContactos;
```

Ao contrário do JavaScript (`contactos.push(...)`) ou do Python (uma tabela da base de dados que cresce livremente), aqui é preciso decidir antecipadamente um número máximo de contactos (`MAX_CONTACTOS`) e reservar, desde logo, espaço fixo (`TAM_NOME`) para cada texto — uma limitação deliberadamente simples, discutida com os alunos como ponto de partida para falar de estruturas de dados mais avançadas.

Trecho 2 — copiar texto em segurança

```
// contacto.c
strncpy(c->nome, nome, TAM_NOME - 1);
c->nome[TAM_NOME - 1] = '\0';
```

Esta dupla instrução evita um dos erros mais comuns (e mais perigosos) em C: escrever texto para além do espaço reservado para ele (*buffer overflow*). `strncpy` copia, no máximo, `TAM_NOME - 1` caracteres, e a linha seguinte garante sempre um caracter final `'\0'` — o símbolo que, em C, marca o fim de um texto.

Trecho 3 — gravar os dados em ficheiro

```
// ficheiros.c
int ficheiros_guardar_contactos(const ListaContactos *lc) {
    FILE *f = fopen(FICHEIRO_CONTACTOS, "wb");
    if (f == NULL) {
        return 0;
    }
    fwrite(&lc->total, sizeof(int), 1, f);
    fwrite(&lc->proximo_id, sizeof(int), 1, f);
    fwrite(lc->itens, sizeof(Contacto), MAX_CONTACTOS, f);
    fclose(f);
    return 1;
}
```

Em vez de uma base de dados (como no TarefaFácil), aqui os dados são gravados diretamente, byte a byte, num ficheiro binário — a forma mais simples (ainda que mais limitada) de persistência de dados em C.

Como compilar e executar

```
cd agendac
make
./agendac
```

Ficheiros entregues

Pasta `agendac/` — ficheiros `.c/.h`, `Makefile`, e instruções completas em `README.md`.

A.5 — Comparação final

	Scratch	ContactosFácil	TarefaFácil	AgendaC
Nível	Inicial	Inicial/intermédio	Intermédio	Avançado
Onde corre	Editor Scratch	Browser	Servidor (Flask)	Linha de comandos
Guarda dados em	Lista, em memória	`localStorage` do browser	Base de dados SQLite	Ficheiros binários
Múltiplos utilizadores	Não	Não	Sim (com login)	Não
Estrutura de dados	Lista do Scratch	Array de objetos (JS)	Tabelas relacionais	`struct` + array de tamanho fixo

Esta tabela é também um bom ponto de partida para uma conversa em grupo: porque é que a mesma ideia — "gerir uma lista de informação" — precisa de soluções tão diferentes em cada

tecnologia? A resposta, quase sempre, tem a ver com o que cada linguagem oferece "de fábrica" e com o que o programador tem de construir manualmente.

Anexo B — Fases do projeto: tabelas e documentos para preencher

Este anexo contém um documento de apoio para cada uma das seis fases do projeto. Recomenda-se que sejam preenchidos ao longo do projeto — e não apenas no final — exatamente como recomendado na secção 9.3 do manual.

Cofinanciado pela União Europeia

B.1 — Ficha de Definição do Projeto (Fase 1)

Nome do projeto: _____

Autor(es): _____

Data: _____ / _____ / _____

1. Qual é o problema ou a necessidade que este projeto resolve?

(Descreva, em poucas frases, a dificuldade real que a aplicação vai resolver, e quem a sente hoje.)

2. Objetivos do projeto (critério SMART — ver secção 4.3)

#	Objetivo (específico, mensurável, atingível, relevante, temporal)
1	
2	
3	

3. Público-alvo

Pergunta	Resposta
Quem vai usar esta aplicação?	
Que conhecimentos técnicos têm?	
Em que dispositivo(s) vai ser usada?	

4. Lista inicial de requisitos (secção 4.5)

#	Requisito	Tipo (funcional / não funcional)
1		
2		

#	Requisito	Tipo (funcional / não funcional)
3		
4		
5		

5. O que este projeto NÃO vai incluir

(Esta lista é a sua principal defesa contra o "scope creep" — secção 1.4 do manual. Sempre que surgir a tentação de acrescentar algo não planeado, volte a esta lista primeiro.)

B.2 — Ficha de Planeamento (Fase 2)

1. Estrutura de tarefas (WBS simplificada — secção 5.2)

#	Tarefa	Depende de (nº)	Estimativa (horas/sessões)
1			
2			
3			
4			
5			
6			
7			
8			

2. Calendário (modelo de Gantt simplificado para as 22 sessões)

Assinale, em cada sessão, as tarefas (pela numeração acima) a realizar.

Tarefa	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18
1																		
2																		
3																		
4																		
5																		

3. Recursos necessários

Recurso	Já disponível? (sim/não)	Observações

4. Riscos identificados

Risco	Probabilidade (baixa/média/alta)	Plano B

5. Critérios de sucesso

(Com base nos objetivos SMART da Ficha de Definição, como vai saber que o projeto foi bem-sucedido?)

#	Critério	Cumprido? (preencher no final)
1		☐ Sim ☐ Não
2		☐ Sim ☐ Não
3		☐ Sim ☐ Não

B.3 — Diário de Bordo do Desenvolvimento (Fase 3)

Preencha uma linha sempre que tomar uma decisão importante, ou sempre que terminar um incremento da fase de Desenvolvimento (secção 6.3).

Data	O que foi feito	Decisões tomadas e porquê	Dificuldades encontradas

B.4 — Ficha de Testes (Fase 4)

1. Testes funcionais (secção 7.2)

#	O que foi testado	Resultado esperado	Resultado obtido	OK?
1				☐
2				☐
3				☐
4				☐
5				☐

2. Testes de casos-limite e de erro (secção 7.3)

#	Situação testada (ex.: campo vazio, acesso sem sessão)	O que aconteceu	Foi corrigido?
1			☐ Sim ☐ Não

#	Situação testada (ex.: campo vazio, acesso sem sessão)	O que aconteceu	Foi corrigido?
2			☐ Sim ☐ Não
3			☐ Sim ☐ Não

3. Teste de usabilidade

Quem testou (não o autor do projeto): _____

O que essa pessoa achou mais confuso ou difícil:

Alterações feitas com base neste teste:

B.5 — Guião de Apresentação (Fase 5)

1. Passos da demonstração (secção 8.2)

Ordem	O que vai ser mostrado	Tempo previsto
1		
2		
3		
4		
5		

2. Problema que o projeto resolve (frase de abertura)

3. Plano B (o que fazer se algo correr mal ao vivo)

4. Perguntas previsíveis e respostas preparadas

Pergunta provável	Resposta preparada

B.6 — Modelo de Relatório Final (Fase 6)

(Estrutura sugerida na secção 9.2 do manual — pode ser usada diretamente como índice do relatório.)

1. Introdução

- Problema e objetivos (retomar a Ficha de Definição, B.1)
- Público-alvo

2. Planeamento

- Como o trabalho foi organizado (retomar a Ficha de Planeamento, B.2)
- Calendário seguido (e eventuais desvios ao plano inicial)

3. Desenvolvimento

- Principais decisões técnicas e porquê (retomar o Diário de Bordo, B.3)
- Tecnologias usadas

4. Testes

- Resumo dos testes realizados (retomar a Ficha de Testes, B.4)
- Problemas encontrados e corrigidos

5. Dificuldades encontradas e como foram resolvidas

- (Secção livre — esta é frequentemente a parte mais valorizada do relatório.)

6. Conclusão e melhorias futuras

- O que ficou concluído, face aos critérios de sucesso definidos (retomar a Ficha de Planeamento, B.2)
- O que faria de forma diferente, ou o que acrescentaria numa versão futura

Anexo C — Sugestões de ferramentas de organização

As ferramentas sugeridas neste anexo não são obrigatórias — o Anexo B já disponibiliza tabelas em papel (ou em documento de texto) que são suficientes para completar o projeto. Estas ferramentas são apenas uma alternativa digital, útil sobretudo para quem vai usar este manual num projeto final mais longo, ou para quem simplesmente prefere organizar o trabalho num computador ou telemóvel. Todas têm uma versão gratuita adequada ao uso de um único projeto escolar; verifique sempre, no momento de as usar, as condições atuais de cada uma, porque os planos gratuitos das ferramentas online vão mudando com o tempo.

As ferramentas estão organizadas pela fase do projeto a que mais se aplicam, e, dentro de cada grupo, da mais simples para a mais completa.

C.1 — Para a fase de Definição e Planeamento

Gestão de tarefas simples

- **Papel e lápis, ou uma folha de cálculo** — nunca é de mais lembrar que, para um projeto desta escala, uma tabela simples (como as do Anexo B) é muitas vezes suficiente, e tem a vantagem de nunca depender de uma conta online.
- **Trello** — organiza tarefas em quadros com colunas (por exemplo, "Por fazer", "Em curso", "Concluído"), de forma muito visual. Adequado a quem está a começar a organizar projetos digitalmente.
- **Notion** — mais completo do que o Trello: permite combinar tarefas, notas e documentos (como o diário de bordo do Anexo B.3) no mesmo espaço. Tem uma curva de aprendizagem um pouco maior.

Calendarização

- **Google Calendar / Calendário do telemóvel** — para marcar prazos e sessões de trabalho, com lembretes automáticos.
- **Uma folha de cálculo com um diagrama de Gantt simples** (Google Sheets ou Excel) — útil para visualizar o calendário de tarefas sugerido na Ficha de Planeamento (Anexo B.2), com mais facilidade de ajustar do que no papel.

C.2 — Para a fase de Elaboração / Desenvolvimento

Desenhar a interface antes de programar (wireframes)

- **Papel e lápis** — continua a ser, muitas vezes, a forma mais rápida de testar uma ideia de interface antes de a construir.
- **Excalidraw** (excalidraw.com) — uma ferramenta de desenho simples e gratuita, com um estilo propositadamente informal ("à mão"), pensada exatamente para esboços rápidos.
- **Figma** — mais usada profissionalmente para desenho de interfaces; tem uma versão gratuita e é uma boa escolha para quem quiser aprofundar esta competência.

Desenhar o modelo de dados ou a estrutura do projeto

- **draw.io** (também conhecido como diagrams.net) — gratuito, corre no browser, e é adequado para desenhar diagramas simples de entidades e relações (como o modelo de dados do TarefaFácil, no Anexo A.3) ou fluxogramas de decisão.

Escrever e organizar código

- **Visual Studio Code (VS Code)** — um editor de código gratuito, usado tanto por principiantes como profissionalmente, com suporte para todas as linguagens dos quatro exemplos deste manual.

- **Repl.it / outros editores online** — permitem escrever e executar código diretamente no browser, sem instalar nada; úteis para começar rapidamente ou para trabalhar em computadores onde não é possível instalar programas.

Guardar versões do código ao longo do projeto

- **Uma pasta com cópias datadas** (por exemplo, `projeto_v1`, `projeto_v2`) — a forma mais simples de nunca perder uma versão anterior que funcionava.
- **Git e GitHub** — a ferramenta usada profissionalmente para guardar o histórico de alterações a um projeto e para o partilhar online. Tem uma curva de aprendizagem real, mas é uma competência valiosa a começar a desenvolver, sobretudo para quem vai usar este manual num projeto final mais avançado (por exemplo, com o exemplo AgendaC, no Anexo A.4).

C.3 — Para a fase de Testes

- **Uma tabela simples** (como a Ficha de Testes do Anexo B.4) — continua a ser a ferramenta mais importante desta fase: o que importa é testar de forma registada, não a ferramenta usada para o registo.
- **Pedir a alguém de fora do projeto para o testar** — não é propriamente uma "ferramenta", mas é provavelmente a técnica mais eficaz de teste de usabilidade (secção 7.2 do manual) ao alcance de qualquer aluno: ver uma pessoa a usar a aplicação por instantes revela, quase sempre, problemas que o próprio autor do projeto já deixou de notar.

C.4 — Para a fase de Apresentação

- **Canva** — gratuito, simples, com modelos prontos para apresentações; adequado a quem quer um resultado visualmente cuidado sem passar muito tempo a desenhar diapositivos.
- **PowerPoint / Google Apresentações** — a opção mais tradicional, com total liberdade de edição.
- **Uma gravação de ecrã** (por exemplo, com a ferramenta de captura já incluída no sistema operativo) — útil como "plano B" discutido na secção 8.2 do manual: ter a demonstração já gravada, para o caso de algo correr mal ao vivo.

C.5 — Para a fase de Relatório

- **Um documento de texto simples** (Word, Google Docs, ou LibreOffice Writer) — suficiente para seguir o modelo do Anexo B.6.

- **Google Docs**, em particular, tem a vantagem de guardar automaticamente o histórico de versões do documento — o que permite recuperar uma versão anterior do relatório, se necessário, sem esforço adicional.

C.6 — Uma nota final sobre ferramentas

A ferramenta mais sofisticada não torna, por si só, um projeto melhor organizado. O que realmente faz a diferença é o hábito — voltar regularmente à Ficha de Definição, atualizar o Diário de Bordo depois de cada sessão de trabalho, marcar os testes à medida que são feitos. Esse hábito é fácil de manter com uma simples folha de papel, e difícil de manter mesmo com a ferramenta digital mais avançada, se não houver disciplina para a usar. Escolha, por isso, a ferramenta mais simples com que se sentir confortável o suficiente para a usar todas as semanas — essa será, quase sempre, a escolha certa.